Microsoft Purview Architecture & Design

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Architecture & Design 5

The problem . . . 5

The solution . . . 5

How it works . . . 5

Why it matters . . . 5

Important boundaries . . . 6

1. Solution Overview — Microsoft Purview Architecture & Design 6

What this solution is . . . 6

Why this matters . . . 7

How it works . . . 7

The five design artifacts (what this solution produces) . . . 8

The phase model (the reference architecture arc) . . . 8

The role-separation model (least privilege, by function) . . . 9

What you can control . . . 10

Limitations (be honest with buyers) . . . 10

Naming convention (used in every doc and script) . . . 10

The standard deployment model . . . 11

2. Requirements Guide — Microsoft Purview Architecture & Design 11

2.1 Licensing baseline and the capabilities it gates . . . 11

2.2 Purview role groups readable, and the separation model agreed . . . 12

2.3 In-scope capabilities and their phase decided (the capability map) . . . 12

2.4 Scoping strategy inputs — adaptive vs static, admin units, workloads . . . 12

2.5 Naming standard adopted . . . 13

2.6 Compliance, records, privacy & legal owners (the determinations) . . . 13

2.7 Security & Compliance PowerShell + Microsoft Graph toolchain . . . 13

2.8 A pilot ring and change control . . . 13

Global Requirements — Microsoft Purview Solutions 14

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 14

2. Purview roles (least-privilege) and portal access . . . 14

3. Security & Compliance PowerShell + Graph toolchain . . . 15

4. Data sources & scope in scope . . . 15

5. Compliance, records, privacy & legal stakeholders . . . 15

6. A pilot scope and change control . . . 15

3. Implementation Guide — Microsoft Purview Architecture & Design 16

Step 1 — Baseline the tenant (Assess) . . . 16

Step 2 — Map the capabilities to licensing and phases (Assess → Design) . . . 17
Step 3 — Design the RBAC model with separation of duties (Design) . . . 17
Step 4 — Design the scoping strategy (Design) . . . 17
Step 5 — Set the naming standard (Design) . . . 18
Step 6 — Stand up the foundation guardrails (Configure — guardrails) . . . 18
Step 7 — Validate the design and report the posture (Validate) . . . 18
Step 8 — Execute the roadmap phase by phase (Expand) . . . 19
5. Testing Guide — Microsoft Purview Architecture & Design 19
Test 1 — Licensing supports every in-scope capability . . . 19
Test 2 — The capability map is complete and phased . . . 20
Test 3 — RBAC separation of duties holds . . . 20
Test 4 — Least-privilege role membership is applied . . . 20
Test 5 — The pilot ring exists and is scoped . . . 20
Test 6 — The scoping strategy is coherent . . . 20
Test 7 — The naming standard is documented and drift is visible . . . 20
Test 8 — Simulation-first discipline is designed in . . . 21
Test 9 — The design posture report is accurate and complete . . . 21
Test 10 — Safe automation defaults . . . 21
6. Troubleshooting — Microsoft Purview Architecture & Design 21
`PURVARCH-Discovery.ps1` shows a capability as unavailable / "CHECK" . . . 21
Role-group names in the design don't exist in the tenant . . . 22
`PURVARCH-Configuration.ps1 -Commit` grants a broad role I didn't intend . . . 22
Separation-of-duties conflicts appear in the RBAC validation . . . 22
The pilot ring group / adaptive scope won't create . . . 22
New capability objects don't conform to the naming standard . . . 23
A capability policy was built without piloting in simulation/test . . . 23
`Connect-IPPSSession` / `Connect-MgGraph` fails . . . 23
The design "drifts" between phases . . . 23
When to escalate beyond this solution . . . 24
Best Practices — Microsoft Purview Architecture & Design 24
1. Map to licensing before you design a phase . . . 24
2. Architect separation of duties from day one . . . 24
3. Design scoping as a first-class decision . . . 24
4. A pilot ring is what makes "pilot" real . . . 25
5. Make simulation-first a designed pattern, not a habit . . . 25
6. Set the naming standard before the first object . . . 25

7. Sequence phases — never run them all at once . . . 25
8. Keep the architecture a living document . . . 25
9. Design carries determinations — it does not make them . . . 25
10. Protect the privacy-sensitive domains in the design itself . . . 25
11. Verify against your tenant — the surfaces evolve . . . 26
12. Keep the design artifacts and evidence intact . . . 26
Deliverables Summary — Microsoft Purview Architecture & Design 26
Documentation . . . 26
PowerShell tools (5) . . . 27
Named objects (verbatim — used across docs, scripts, and tests) . . . 28
What this solution deliberately does NOT do . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary — Microsoft Purview Architecture & Design

The problem

Microsoft Purview is not one product — it is a suite of compliance and data-governance capabilities (labels, DLP, retention and records, insider risk, communication compliance, audit, data security posture, data governance) that all share the **same tenant, roles, scopes, and naming**. Turned on ad hoc — a label here, a DLP policy there — Purview **sprawls**: overlapping policies, over-privileged administrators, inconsistent names, no scoping discipline, and no defensible answer to "what is deployed, and why?" That is a compliance gap and a data-risk exposure at once, and it gets **more** expensive to fix with every policy added on top of the mess.

The solution

This solution delivers the **architecture and design that comes before you scale Purview**:

- **A capability map** — which capabilities are in scope, tied to the **licensing** that gates each one

(so you never design a phase around a feature you cannot license) and the **phase** it belongs to.

- **An RBAC model** — least-privilege **role groups** with **separation of duties**, so information

protection, insider risk, eDiscovery, and records are administered by **separate** people from day one.

- **A scoping strategy** — adaptive vs static scopes, administrative units, the in-scope workloads, and

a named **pilot ring** that keeps every high-impact policy contained until it is validated.

- **A naming standard** — a `<Purpose> – <Scope> – <Stage>` convention every future object follows, so

the estate stays legible as it grows.

- **A phased roadmap** — Foundation → Know → Protect → Prevent loss → Govern → Advanced, sequenced with

entry/exit gates so each phase builds on the last.

How it works

We follow a repeatable model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. We baseline the tenant, map capabilities to licensing and phases, design the RBAC model, scoping strategy, and naming standard, and stand up the **foundation guardrails** (least-privilege roles, a pilot ring, naming) — then each capability is built **against this architecture**, and every high-impact policy is **piloted in simulation/test first**. The solution ships as **self-guided documentation plus five PowerShell tools** (Security & Compliance PowerShell / Graph) that assess the estate, scaffold the RBAC and scope guardrails, validate the design, and produce a posture report — all **read-only or `-WhatIf` by default**, with **no credentials in code**.

Why it matters

• **Design once, or refactor forever** — the expensive decisions (roles, scopes, naming) are made once,

deliberately, so every later capability slots in cleanly instead of triggering a re-architecture.

• **Least privilege, architected in** — separation of duties from day one, not bolted on after an audit finds everyone holding a broad role.

• **Contained by design** — a pilot ring and simulation-first pattern make "pilot" a defined population, not a hope, for every high-impact policy.

• **A fraction of consulting cost** — the same disciplined method a specialist would use to plan a Purview program, delivered as a repeatable, tested runbook.

Important boundaries

This solution designs the **architecture** and stands up the **foundation guardrails**; each capability's policies (labels, DLP, retention, insider risk, etc.) are built in **that capability's own solution**, against this design, and piloted in simulation/test first. Purview capabilities are **licensing-dependent** (many require **Microsoft 365 E5 / E5 Compliance**; some are in preview) and the portal is **actively evolving** — capability availability, role-group names, and paths must be **verified against your licensing and tenant**. And the **determinations** the architecture carries — regulatory applicability, retention schedules, records file plans, and the lawful basis for insider-risk or communication monitoring — are **compliance / records / privacy / legal** decisions for your organization, not outputs of this design.

1. Solution Overview — Microsoft Purview Architecture & Design

What this solution is

Microsoft Purview is not one product — it is a **suite** of compliance and data-governance capabilities (sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance) that all share the same **tenant, roles, scopes, and naming**. Turn them on ad hoc — a label here, a DLP policy there — and you get **sprawl**: overlapping policies, over-privileged administrators, inconsistent names, no scoping discipline, and no way to say what is deployed or why. That is a compliance gap and a data-risk exposure at the same time.

This solution delivers the **architecture and design that comes *before* you scale Purview**: a **capability map** (which Purview capabilities are in scope, mapped to the licensing that gates them and the phase they belong to), an **RBAC model** (least-privilege role groups with separation of duties), a **scoping strategy** (adaptive scopes, administrative units, pilot rings, and the workloads/data sources in scope), a **naming standard** every future policy/label/object follows, and a **phased roadmap** that sequences the capabilities so each phase builds on the last. It is delivered with the house model — **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand** — using only the **Microsoft Purview portal**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph**. No third-party tooling.

> ***This is the design layer, not the individual capability build.*** *This solution produces the* ***reference architecture*** *— the capability map, RBAC model, scoping plan, naming standard, and phase roadmap*

*— and stands up the **foundation guardrails** (least-privilege roles, scopes, naming, a pilot ring). Building out each capability (a specific DLP policy, a label taxonomy, a retention schedule, an insider-risk policy) is done in that capability's own solution, **against this design**. Every high-impact policy that design instantiates is **piloted in simulation/test first** under its own change control — this solution defines that as the standing pattern.*

Why this matters

• **Design once, or refactor forever.** Role groups, scopes, and naming set early are cheap; changing them after fifty policies reference them is expensive and risky. A reference architecture makes the expensive decisions **once, deliberately**, so every later capability slots in cleanly.

• **Least privilege has to be architected, not bolted on.** Purview's power is fine-grained role groups; that only protects you if administrators are **separated by function** (information protection ≠ insider risk ≠ eDiscovery ≠ records) from day one. Retrofitting separation of duties after everyone is a Compliance Administrator is painful and audit-visible.

• **Scoping is what keeps a pilot a pilot.** Adaptive scopes, administrative units, and a named pilot ring are what let you validate a high-impact policy on **a defined population** before it touches the tenant. Without a scoping design, "pilot" means "hope."

• **Licensing gates the map.** Most Purview capabilities require **E5 / E5 Compliance** (some are in preview). A capability map that ties each capability to the plan that enables it prevents designing a phase around a feature you cannot license — the most common Purview planning failure.

• **The determinations are not technical.** *Which* regulations apply, *what* the retention schedule is, *whether* insider-risk monitoring is lawful, *what* the records file plan contains — these are **compliance / records / privacy / legal** determinations. This solution designs the technical architecture that will carry them; it does not make them.

How it works

1. **Baseline the tenant (Assess).** Inventory what already exists — licensing/SKUs and which capabilities they gate, current role groups and their membership, existing labels/DLP/retention policies and scopes, and the audit state — as the `PA Design Baseline – Tenant – Assess` starting point (`PURVARCH-Discovery.ps1`, read-only).

1. **Map the capabilities to licensing and phases (Assess → Design).** With stakeholders, decide which Purview capabilities are **in scope**, confirm each is **licensable** on your plan, and assign each to a **phase** — the `PA Capability Map – Tenant – Design` and `PA Phase Roadmap – Program – Design`.

1. **Design the RBAC model (Design).** Define the least-privilege **role groups** per capability with **separation of duties** (who administers vs analyzes vs investigates vs reads), the `PA RBAC Model – Tenant – Design`, and identify conflicts (`PA RBAC SoD – Conflicts`).

1. **Design the scoping strategy (Design).** Decide **adaptive scopes** vs static scopes, whether

administrative units are used, the in-scope **workloads/data sources**, and the named **pilot ring** — the `PA Scoping Plan – Tenant – Design`, with the pilot group `SG-PA-Pilot` and example adaptive scope `AS-PA-Pilot`.

1. **Set the naming standard (Design).** Adopt the `<Purpose> – <Scope> – <Stage>` convention every

future label/policy/scope/object follows — the `PA Naming Standard – Tenant – Design` — so the estate stays legible as it grows.

1. **Stand up the foundation guardrails (Configure — guardrails).** Apply the designed **role-group

membership **(least privilege, separated) and scaffold the** pilot ring / adaptive scope** — the only things this solution *configures*; every capability policy is built later, against the design, in simulation first (`PURVARCH-Configuration.ps1`, `PURVARCH-Set-Scope.ps1`, `-WhatIf` default).

1. **Validate and hand off (Validate → Expand).** Confirm the design is conformant — capabilities

licensable, roles separated, scopes in place, naming applied — with `PURVARCH-Validation.ps1` and the `PA Design Posture – Report` (`PURVARCH-Report.ps1`), then execute the roadmap phase by phase, each capability piloted in simulation/test against this architecture.

The five design artifacts (what this solution produces)

• **Capability map** — every in-scope Purview capability, the **licensing** that gates it (E3 / E5 / E5

Compliance / preview / add-on), its **phase**, its **owner**, and its status (in scope / deferred — not licensed / out of scope). It is the single answer to "what are we deploying, and can we?"

• **RBAC model** — the **role groups** per capability, mapped to a **least-privilege, separated** set of

administrator / analyst / investigator / reader duties, plus the separation-of-duties rules and any conflicts. This is the auditable access design.

• **Scoping plan** — **adaptive vs static** scopes, **administrative unit** use, the in-scope

workloads and data sources, and the **pilot ring** — the mechanism that keeps every high-impact policy contained until it is validated.

• **Naming standard** — the `<Purpose> – <Scope> – <Stage>` convention (e.g. `DLP-PII-Sim`,

`Label-Confidential-Prod`, `AS-Finance-Prod`) so the estate is legible and searchable as it scales.

• **Phase roadmap** — the sequence (Foundation → Know → Protect → Prevent loss → Govern → Advanced),

what each phase delivers, its entry/exit gates, and its dependency on the phase before it.

The phase model (the reference architecture arc)

Phase	Theme	Representative capabilities (all built later, in simulation first)
0 — Foundation & Governance	Roles, scopes, naming, audit	Purview RBAC role groups, adaptive scopes / administrative units, unified audit log, naming standard

Phase	Theme	Representative capabilities (all built later, in simulation first)
1 — Know your data	Classification	Sensitive information types (SITs), trainable classifiers, Content/Activity explorer, data classification
2 — Protect your data	Labeling	Sensitivity labels + label policies, encryption, container/site labeling, auto-labeling
3 — Prevent data loss	DLP	DLP across Exchange/SharePoint/OneDrive/Teams and **endpoint** — **simulation-first**
4 — Govern & retain	Lifecycle & records	Retention labels/policies, records management, data lifecycle, disposition
5 — Manage risk (advanced)	Insider & AI	Insider Risk Management, Communication Compliance, Adaptive Protection, DSPM, Copilot/AI data governance

*Phases are **sequenced, not simultaneous** — each depends on the guardrails and classification the earlier phases establish. Capability availability per phase is **licensing-gated and evolving**; the capability map confirms each is licensable **before** the phase is planned around it. **Verify availability and paths against your own licensing and tenant.***

The role-separation model (least privilege, by function)

Capability domain	Administer	Analyze / investigate	Read-only
Information Protection (labels/DLP)	Information Protection Admins	Information Protection Analysts / Investigators	Information Protection Readers
Data Lifecycle & Records	Records Management	—	Global Reader / read role
Insider Risk Management	Insider Risk Management Admins	Analysts / Investigators	Insider Risk Management Auditors
Communication Compliance	Communication Compliance Admins	Analysts / Investigators	Communication Compliance Viewers
eDiscovery	eDiscovery Administrator	eDiscovery Manager	reviewer (case-scoped)
Audit / posture	—	Audit roles	Global Reader

*Global Administrator and Compliance Administrator subsume many of these — reserve them and prefer the **narrow, function-specific** role group. **Verify the exact role-group names and rights in your tenant** — Purview role naming (and the transition to unified Purview RBAC "roles & scopes") is **evolving**; do not assume. Separation of duties is the point: information protection ≠ insider risk ≠ eDiscovery ≠ records, and readers/auditors stay read-only.*

What you can control

- The **capability map** — which capabilities are in scope, and their phase, owner, and status.
- The **RBAC model** — the role groups, the least-privilege separation, and the SoD rules.
- The **scoping strategy** — adaptive vs static scopes, administrative-unit use, the in-scope workloads, and the pilot ring.
- The **naming standard** applied to every future object.
- The **phase roadmap** — sequence, gates, and dependencies — and the **pilot ring** validated first.

Limitations (be honest with buyers)

- **It designs; it does not build every capability.** This solution produces the architecture and stands up the **foundation guardrails** (roles, scopes, naming, pilot ring). Each capability's policies (labels, DLP, retention, insider risk, etc.) are built in **that capability's own solution**, against this design, and **piloted in simulation/test first**.
- **Licensing gates the map.** Advanced capabilities require **E5 / E5 Compliance** (or add-ons); some are in **preview**. A capability you design a phase around may not be licensable — the map flags it as **deferred (not licensed)**. **Verify entitlement — do not assume.**
- **Portal-driven and evolving.** Role groups, adaptive scopes, administrative units, and the unified Purview RBAC surface are managed in the **portals** and are **actively changing**. The scripts assess, scaffold, validate, and report; they do not replace the portal, and they flag where a surface is portal-only. **Verify paths against your tenant.**
- **Determinations are deferred.** Regulatory applicability, retention schedules, records file plans, and the lawful basis for insider-risk monitoring are **compliance / records / privacy / legal** determinations. The architecture carries them; it does not make them.
- **No secrets in automation.** The provided scripts use interactive modern-auth sign-in; they never embed credentials, and default to read-only or `-WhatIf`.

Naming convention (used in every doc and script)

Every design artifact and every object this architecture governs is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and portal. The standard set:

Artifact	Name
Tenant baseline snapshot	`PA Design Baseline – Tenant – Assess`
Capability map	`PA Capability Map – Tenant – Design`
RBAC model	`PA RBAC Model – Tenant – Design`
RBAC separation-of-duties conflicts	`PA RBAC SoD – Conflicts`
Scoping plan	`PA Scoping Plan – Tenant – Design`

Artifact	Name
Naming standard	`PA Naming Standard – Tenant – Design`
Phase roadmap	`PA Phase Roadmap – Program – Design`
Design posture report	`PA Design Posture – Report`
Pilot ring group	`SG-PA-Pilot`
Example adaptive scope (pilot)	`AS-PA-Pilot`

> *Future capability objects follow the same pattern, e.g.* `DLP-PII-Sim`, `Label-Confidential-Prod`, `Retention-Finance-Prod`, `AS-Finance-Prod` — `<Purpose> – <Scope> – <Stage>`, *with* `Sim` *for simulation/test and* `Prod` *for enforced.*

The standard deployment model

> *Baseline the tenant → map capabilities to licensing and phases → design the RBAC model, scoping strategy, and naming standard → stand up the foundation guardrails (least-privilege roles, pilot ring/adaptive scope, naming) → validate the design and report the posture → execute the roadmap phase by phase, each capability piloted in simulation/test against this architecture. Pattern:* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot (simulate/test)*** *→* ***Validate*** *→* ***Expand*** *— regulatory, records, and legal determinations deferred to compliance/records/privacy/legal at every step.*

2. Requirements Guide — Microsoft Purview Architecture & Design

Verify **every** requirement before designing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant licensing, Purview roles, the PowerShell/Graph toolchain, in-scope data sources, compliance/privacy/legal stakeholders, and a pilot scope with change control) are in `global-requirements-purview.md` — this guide adds the **architecture-and-design-specific** detail. **Licensing, capability availability, role-group names, and portal paths are plan-dependent and the portal evolves — verify each against your own licensing and tenant.**

2.1 Licensing baseline and the capabilities it gates

Description: Confirm the tenant's compliance licensing — **E3** (baseline: manual sensitivity/ retention labels, basic DLP, core audit) versus **Microsoft 365 E5 / E5 Compliance** (advanced DLP, auto-labeling, trainable classifiers, insider risk, communication compliance, records management, Audit Premium, DSPM, adaptive protection) and any **add-ons** — because the licensing is what determines which capabilities can even appear on the capability map. **Why:** Designing a phase around a capability the plan does not license is the most common Purview planning failure. The capability map must tie each capability to the entitlement that enables it. **Verify:** Microsoft 365 admin center → `Billing → Licenses` for the E5 / E5 Compliance / add-on SKUs; in the Purview portal confirm the capability's node is visible (not "upgrade required"). `PURVARCH-Discovery.ps1` reports the subscribed SKUs it can read via Graph and maps them to an **indicative** capability-availability list. **Confirm exact entitlement with Microsoft licensing — the script's mapping is indicative, not authoritative. Implement:** Record the licensing

baseline; where a needed capability is unlicensed, mark it **deferred (not licensed)** on the map rather than designing around it, or plan the license change.

2.2 Purview role groups readable, and the separation model agreed

Description: The current Purview / compliance **role groups** and their membership are readable, and stakeholders agree the **separation-of-duties** intent — information protection, insider risk, communication compliance, eDiscovery, records, and audit administered by **separate** people, with Global Administrator and Compliance Administrator reserved. **Why:** The RBAC model is the auditable access design; it can only be built from the current state and an agreed separation intent. Retrofitting separation after everyone holds a broad role is expensive and audit-visible. **Verify:** Purview portal → `Settings → Roles & scopes → Permissions` (verify path — the unified Purview RBAC surface is evolving) or, via `Connect-IPPSSession`, `Get-RoleGroup` / `Get-RoleGroupMember`. `PURVARCH-Discovery.ps1` prints each role group's membership and `PURVARCH-Validation.ps1` flags separation-of-duties conflicts (e.g. one person administering both insider risk and eDiscovery) as the `PA RBAC SoD – Conflicts` register. **Verify the exact role-group names in your tenant — they evolve. Implement:** Capture the current membership and the agreed separation model before applying the RBAC design with `PURVARCH-Configuration.ps1` (`-WhatIf` first, then `-Commit`), granting the **minimum** role each function needs.

2.3 In-scope capabilities and their phase decided (the capability map)

Description: The **set of Purview capabilities** in scope — from the phase model (Foundation → Know → Protect → Prevent loss → Govern → Advanced) — with each capability assigned a **phase**, an **owner**, and a **status** (in scope / deferred — not licensed / out of scope), driven by the organization's compliance obligations. **Why:** The capability map is the unit of the whole design; without agreed scope and sequencing, the RBAC model and roadmap have nothing to key off, and phases collide. **Verify:** The capability list, phases, owners, and statuses are agreed with the governance owners and each in-scope capability is confirmed licensable (Requirement 2.1). `PURVARCH-Discovery.ps1` / `PURVARCH-Report.ps1` emit the `PA Capability Map – Tenant – Design` scaffold from what it can read. **Implement:** Run the capability-scoping workshop, confirm licensing per capability, and record the map and `PA Phase Roadmap – Program – Design` before designing RBAC and scoping.

2.4 Scoping strategy inputs — adaptive vs static, admin units, workloads

Description: The inputs to the scoping design: whether **adaptive scopes** (query-based: user, site, or group membership) or **static scopes** are preferred per capability, whether **administrative units** (Entra) are used to delegate scoped administration, the **workloads / data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate), and the named **pilot ring** to validate on first. **Why:** Scoping is what keeps a high-impact policy contained to a defined population until it is validated. Adaptive scopes and admin units are architectural choices that every later policy inherits — they must be designed, not improvised. **Verify:** The preferred scope type per capability, admin-unit use, in-scope workloads, and the pilot population are recorded (`PA Scoping Plan – Tenant – Design`). `PURVARCH-Discovery.ps1` inventories existing adaptive scopes (`Get-AdaptiveScope`, where available) and reports readable workloads/managed devices via Graph. **Implement:** Decide scope type per capability and the pilot ring; scaffold the pilot group `SG-PA-Pilot` and (where adaptive scopes are used) the example `AS-PA-Pilot` with `PURVARCH-Set-Scope.ps1`

(`-WhatIf` first). Adaptive-scope availability is licensing-gated — **verify in your tenant**.

2.5 Naming standard adopted

Description: A **naming standard** — the `<Purpose> – <Scope> – <Stage>` convention — adopted before any capability policy/label/scope is created, with `Sim` for simulation/test and `Prod` for enforced (e.g. `DLP-PII-Sim`, `Label-Confidential-Prod`, `AS-Finance-Prod`). **Why:** Names are effectively immutable once dozens of policies reference them. A standard set up front keeps the estate legible, searchable, and auditable as it scales across phases. **Verify:** The standard is documented (`PA Naming Standard – Tenant – Design`) and agreed with the capability owners; `PURVARCH-Validation.ps1` reports existing objects that do **not** conform so drift is visible. **Implement:** Adopt the standard and record it before Phase 0 completes; apply it to every object from Phase 1 onward.

2.6 Compliance, records, privacy & legal owners (the determinations)

Description: Named owners in **compliance, records-management, privacy, security, and legal** who make the determinations the architecture will carry — **regulatory applicability**, **retention schedules**, **records file plans**, the **lawful basis** for insider-risk/communication monitoring — and who own each capability domain in the RBAC model. **Why:** The architecture is technical; the determinations it must satisfy are not. Several capabilities (insider risk, communication compliance) are HR/privacy-sensitive and require role separation and a lawful basis established **before** they are built. **Verify:** The applicable frameworks, the named approvers per capability domain, and the privacy sensitivities are recorded and reflected in the RBAC model and roadmap. **Implement:** Engage the governance functions during the design workshop; capture the obligations and the domain owners before finalizing the RBAC model and phase roadmap.

2.7 Security & Compliance PowerShell + Microsoft Graph toolchain

Description: A workstation with the **ExchangeOnlineManagement** module (providing `Connect-IPPSSession` for role-group, policy, and adaptive-scope cmdlets) and **Microsoft Graph PowerShell** (for licensing/SKU, administrative-unit, and workload/device signals), used with a modern-auth / MFA-capable admin holding the appropriate read roles. **Why:** The five provided scripts assess the estate, scaffold the RBAC and scope guardrails, validate the design, and report — through Security & Compliance PowerShell and Graph. The architecture itself is **portal-managed**; the scripts support it, they do not replace it. **Verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed; `Get-RoleGroup` returns the compliance role groups and `Get-MgSubscribedSku` returns the licensing. `PURVARCH-Discovery.ps1` performs these checks. **Implement:** `Install-Module ExchangeOnlineManagement`; `Install-Module Microsoft.Graph`; sign in with the read-role-holding admin and consent the **read** scopes the scripts request.

2.8 A pilot ring and change control

Description: A named **pilot ring** (`SG-PA-Pilot`, a defined set of users/sites/groups) to validate the foundation guardrails and every subsequent capability on **before** tenant-wide expansion, plus a **change-control** path — consistent with `global-requirements-purview.md` §6 — recognizing that most Purview policies support a **simulation / test / disabled** mode that MUST precede enforcement. **Why:** Every phase and every capability follows **Assess** → **Design** → **Configure** → **Pilot (simulate)** → **Validate** → **Expand**; the pilot ring is the mechanism that makes "pilot" a defined population rather than a

hope. DLP, labels, retention, and insider-risk changes are high-impact. **Verify:** A named pilot ring and its owner exist; a change window/approval path is defined; the scoping plan references the ring. **Implement:** Define the pilot ring and scaffold `SG-PA-Pilot` (`PURVARCH-Set-Scope.ps1 -WhatIf` first), agree change control, and validate the foundation on the ring before executing Phase 1.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Architecture & Design

Follow these steps **in order**. Design work is done in the **Microsoft Purview portal** (`https://purview.microsoft.com`), the **Microsoft 365 admin center**, and, where automated, via **Security & Compliance PowerShell** (`Connect-IPPSSession`, for role groups, policies, and adaptive scopes) and **Microsoft Graph** (for licensing/SKU, administrative units, and workload signals). Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are reused verbatim. **Portal paths, capability availability, role-group names, and cmdlets are plan-dependent and the portal evolves — verify against your tenant.**

> ***Model:*** *Baseline the tenant, map capabilities to licensing/phases, design the RBAC model + scoping strategy + naming standard, stand up the foundation guardrails, validate the design, then execute the roadmap phase by phase. Pattern:* ***Assess → Design → Configure → Pilot (simulate/test) → Validate → Expand. This solution designs the architecture and the foundation guardrails; each capability's policies are built later, against this design, and piloted in simulation/test first.*** *Regulatory, records, and legal* ***determinations*** *are deferred to compliance/records/privacy/legal throughout.*

Step 1 — Baseline the tenant (Assess)

1. Confirm licensing (Requirement 2.1): Microsoft 365 admin center → `Billing → Licenses` for the E3 / E5 / E5 Compliance / add-on SKUs. **Verify entitlement with Microsoft licensing — do not assume.**

1. Run `PURVARCH-Discovery.ps1` (read-only). It reports: the subscribed SKUs it can read and an **indicative** capability-availability mapping; the current **role groups** and their membership; the existing **labels, DLP, and retention policies** and any **adaptive scopes**; and the **audit** state. Save this as the `PA Design Baseline – Tenant – Assess` snapshot.

1. **Pass/fail gate:** the baseline is captured — licensing, roles, existing policies/scopes, audit state — and the governance owners (Requirement 2.6) are engaged. **Do not proceed** to design until the baseline is agreed. Fail → complete the inventory first.

Step 2 — Map the capabilities to licensing and phases (Assess → Design)

1. With the governance owners, decide which Purview capabilities are **in scope** (from the phase model:

Foundation → Know → Protect → Prevent loss → Govern → Advanced), and for each confirm it is **licensable** on your plan (Requirement 2.1).

1. Assign each capability a **phase**, an **owner**, and a **status** (in scope / **deferred — not

licensed** / out of scope). Use `PURVARCH-Report.ps1` to emit the `PA Capability Map – Tenant – Design` scaffold from the discovery data; complete it with the owners' decisions.

1. Sequence the phases into the `PA Phase Roadmap – Program – Design`, recording each phase's entry/exit

gate and its dependency on the phase before it.

1. **Pass/fail gate:** every in-scope capability is licensable and assigned a phase and owner;

unlicensed capabilities are recorded as **deferred**, not designed around. **Applicability is the owners' determination**, not a technical default. Fail → resolve licensing/scope before designing RBAC.

Step 3 — Design the RBAC model with separation of duties (Design)

1. From the baseline (Step 1) and the agreed separation intent (Requirement 2.2), design the

least-privilege **role groups per capability domain** — Information Protection, Data Lifecycle & Records, Insider Risk Management, Communication Compliance, eDiscovery, and Audit — mapping each to **administer / analyze-investigate / read-only** duties. Reserve Global Administrator and Compliance Administrator.

1. Record the design as `PA RBAC Model – Tenant – Design`, and run `PURVARCH-Validation.ps1` to produce

the `PA RBAC SoD – Conflicts` register — e.g. a person administering **both** insider risk and eDiscovery, or a reader who also holds a broad admin role.

1. **Pass/fail gate:** each capability domain has named least-privilege role groups; separation of

duties holds (information protection ≠ insider risk ≠ eDiscovery ≠ records); conflicts are recorded and accepted or resolved. **Verify the exact role-group names in your tenant.** Fail → resolve conflicts before applying.

Step 4 — Design the scoping strategy (Design)

1. Decide, per capability, **adaptive** vs **static** scope (Requirement 2.4): adaptive scopes are

query-based (user attributes, site properties, or group membership) and re-evaluate automatically; static scopes are explicit. Decide whether **administrative units** delegate scoped administration.

1. Confirm the in-scope **workloads/data sources** and define the **pilot ring** — a named population to

validate every capability on first. Record all of this as `PA Scoping Plan – Tenant – Design`.

1. **Pass/fail gate:** each capability has a chosen scope type, the in-scope workloads are agreed, and a

named pilot ring exists. Adaptive-scope availability is licensing-gated — **verify in your tenant**. Fail → complete the scoping plan before standing up guardrails.

Step 5 — Set the naming standard (Design)

1. Adopt the `<Purpose> – <Scope> – <Stage>` convention (Requirement 2.5) — `Sim` for simulation/test,

`Prod` for enforced — as the `PA Naming Standard – Tenant – Design`, and agree it with every capability owner.

1. Run `PURVARCH-Validation.ps1` to list existing objects that do **not** conform, so drift is visible

and can be planned into remediation (renaming is done in the portal per object).

1. **Pass/fail gate:** the standard is documented and agreed; nonconforming existing objects are

identified. Fail → finalize the standard before creating any new object.

Step 6 — Stand up the foundation guardrails (Configure — guardrails)

1. **Apply the RBAC model.** Portal → `Settings → Roles & scopes → Permissions` (verify path): populate

the designed role groups with least-privilege, separated membership. Or manage membership with `PURVARCH-Configuration.ps1` — it composes `Add-RoleGroupMember` / `Remove-RoleGroupMember`, **runs with** `-WhatIf` **by default** and prints exactly what it would change; add `-Commit` to apply.

1. **Scaffold the pilot ring and (where used) an adaptive scope.** Use `PURVARCH-Set-Scope.ps1` to

preview and (with `-Commit`) create/populate the pilot group `SG-PA-Pilot`, and — where adaptive scopes are in the design — scaffold the example `AS-PA-Pilot` (`New-AdaptiveScope`, where available in your tenant). `-WhatIf` **by default.**

1. **This is the only thing this solution configures.** No capability policy (DLP, label, retention,

insider risk) is created here — those are built in their own solutions, **against this design**, and **piloted in simulation/test first**. The naming standard and pilot ring are what make that pilot discipline enforceable.

1. **Pass/fail gate:** role groups hold the least-privilege, separated membership; the pilot ring (and

any adaptive scope) exists; re-run `PURVARCH-Discovery.ps1` and confirm. **Rollback:** remove any over-granted membership with `PURVARCH-Configuration.ps1 -Commit`; remove the pilot group/scope in the portal — no capability policy references it yet, so removal is safe.

Step 7 — Validate the design and report the posture (Validate)

1. Run the Testing Guide (`05-testing.md`). Confirm: every in-scope capability is licensable; the RBAC

model is applied with separation holding and no unaccepted conflicts; the pilot ring/adaptive scope exists; the naming standard is documented and drift is identified.

1. Run `PURVARCH-Validation.ps1` (read-only) for the conformance checks and `PURVARCH-Report.ps1` for

the `PA Design Posture – Report` (capability coverage vs licensing, RBAC separation, scope coverage, naming conformance, phase status).

1. **Pass/fail gate:** the design is conformant and the posture report is accurate. Fail → correct the guardrails and re-validate before executing the roadmap.

Step 8 — Execute the roadmap phase by phase (Expand)

1. Begin **Phase 0/1** and proceed in sequence. For each capability, its **own solution** builds the policy **against this architecture** — using the designed role groups, the scoping plan, the pilot ring, and the naming standard — and **pilots it in simulation/test first**, passes that capability's gates, then expands. This solution's design is the **contract** every phase builds to.

1. At each phase boundary, re-run `PURVARCH-Validation.ps1` / `PURVARCH-Report.ps1` to confirm the new objects conform to the RBAC model, scoping plan, and naming standard, and to update the phase status on the roadmap.

1. Maintain the architecture as a living document: review the RBAC model on joiners/leavers, re-check licensing when plans change, watch for **portal/RBAC evolution** Microsoft ships, and re-baseline periodically.

1. **Pass/fail gate:** each completed phase's objects conform to the design; separation of duties still holds; the roadmap status is current. **Rollback:** a phase's capability policies are governed by their own solution's rollback; the architecture itself can be narrowed (scope, phase) at any time without deleting completed, conformant work.

What this guide does NOT do

• **Build the capability policies.** DLP policies, label taxonomies, retention schedules, insider-risk policies, etc. are built in **their own solutions**, against this design, and **piloted in simulation/test first**. This guide designs the architecture and stands up the foundation guardrails.

• **Make the determinations.** Regulatory applicability, retention schedules, records file plans, and the lawful basis for insider-risk/communication monitoring are **compliance/records/privacy/legal** determinations. The architecture carries them; it does not decide them.

5. Testing Guide — Microsoft Purview Architecture & Design

Run these after designing the capability map, RBAC model, scoping strategy, and naming standard (Implementation Steps 2–5) and standing up the foundation guardrails (Step 6), and again at each phase boundary as the roadmap executes. Every test here is **safe and read-only** or contained to the **foundation guardrails** (role-group membership and the pilot ring) — no capability policy is enforced by this solution. The point is to prove the **design is conformant** — capabilities licensable, roles separated, scopes in place, naming applied — before capabilities are built against it. **Applicability, retention schedules, and lawful basis remain compliance/records/privacy/legal determinations — these tests validate the architecture, not the determinations.**

Test 1 — Licensing supports every in-scope capability

Do: Run `PURVARCH-Discovery.ps1`; compare its subscribed-SKU / capability-availability output to the `PA Capability Map – Tenant – Design`. **Expected:** Every capability marked **in scope** is licensable on the plan; capabilities that are not are marked **deferred (not licensed)**, not designed into a phase. **Fail** → correct the map or the licensing (Requirement 2.1) before proceeding. **The script mapping is indicative — confirm entitlement with Microsoft licensing.**

Test 2 — The capability map is complete and phased

Do: Review the `PA Capability Map – Tenant – Design` and `PA Phase Roadmap – Program – Design`. **Expected:** Every in-scope capability has an **owner**, a **phase**, and a **status**; phases are sequenced with entry/exit gates and dependencies. **Fail** → complete the map/roadmap with the governance owners (Implementation Step 2).

Test 3 — RBAC separation of duties holds

Do: Run `PURVARCH-Validation.ps1`; review the `PA RBAC SoD – Conflicts` register against the `PA RBAC Model – Tenant – Design`. **Expected:** Each capability domain has named least-privilege role groups; no person administers two **separated** domains (e.g. insider risk **and** eDiscovery) unless the owners explicitly accepted it; readers/auditors are read-only; Global Administrator / Compliance Administrator are reserved, not the default. **Fail** → resolve conflicts with `PURVARCH-Configuration.ps1` (Implementation Step 6) before building capabilities.

Test 4 — Least-privilege role membership is applied

Do: Run `PURVARCH-Discovery.ps1`; review each designed role group's membership. **Expected:** Each function holds the **minimum** role it needs; nobody holds a broad role that subsumes a function they should not have. **Fail** → correct membership with `PURVARCH-Configuration.ps1` (`-WhatIf` then `-Commit`).

Test 5 — The pilot ring exists and is scoped

Do: Run `PURVARCH-Set-Scope.ps1` with no action switch (dry-run) and review the `SG-PA-Pilot` membership; where adaptive scopes are used, confirm `AS-PA-Pilot` exists. **Expected:** A named pilot ring (`SG-PA-Pilot`) exists with the intended population, and any designed adaptive scope is present — the mechanism that will contain every high-impact capability policy to a defined population before enforcement. **Fail** → scaffold the ring/scope (Implementation Step 6). Adaptive-scope availability is licensing-gated — **verify in your tenant**.

Test 6 — The scoping strategy is coherent

Do: Review the `PA Scoping Plan – Tenant – Design` against the capability map. **Expected:** Each capability has a chosen **adaptive vs static** scope, administrative-unit use is decided, and the in-scope workloads/data sources are agreed — so every later policy inherits a defined scope. **Fail** → complete the scoping plan (Implementation Step 4).

Test 7 — The naming standard is documented and drift is visible

Do: Run `PURVARCH-Validation.ps1`; review the list of existing objects that do **not** conform to `PA Naming Standard – Tenant – Design`. **Expected:** The `<Purpose> – <Scope> – <Stage>`

standard (with `Sim` / `Prod`) is documented and agreed; nonconforming existing objects are identified for planned remediation. **Fail** → finalize the standard and record the drift (Implementation Step 5).

Test 8 — Simulation-first discipline is designed in

Do: Confirm the design and roadmap state that every high-impact capability policy (DLP, labels, retention, insider risk) is created in **simulation/test/disabled** mode first, named with the `Sim` suffix, and scoped to `SG-PA-Pilot`. **Expected:** The pilot-in-simulation pattern is an explicit, named part of the architecture, not an afterthought — the pilot ring and naming standard make it enforceable. **Fail** → add the pattern to the roadmap and naming standard before executing any phase.

Test 9 — The design posture report is accurate and complete

Do: Run `PURVARCH-Report.ps1` for the `PA Design Posture – Report`. **Expected:** The report rolls up capability coverage vs licensing, the RBAC separation snapshot, scope coverage, naming conformance, and phase status — matching the portal and the design artifacts. It notes that the **authoritative** role/scope/policy state lives in the Purview portal. **Fail** → confirm the inputs and re-run.

Test 10 — Safe automation defaults

Do: Run `PURVARCH-Configuration.ps1` and `PURVARCH-Set-Scope.ps1` with no action switch. **Expected:** Each runs in `-WhatIf` mode and only *describes* the change; nothing is written until `-Commit` is supplied. `PURVARCH-Discovery.ps1`, `PURVARCH-Validation.ps1`, and `PURVARCH-Report.ps1` are read-only. **Fail** → stop; do not run with `-Commit` until the dry-run output is correct.

Gate to execute the roadmap: Tests 1–2 (licensing + map), 3–4 (RBAC separation + least privilege), 5–6 (scoping + pilot ring), 7–8 (naming + simulation-first), and 9–10 (reporting + safety) all pass → begin Phase 0/1 (Implementation Step 8), each capability built against this design and piloted in simulation/test. Any fail → correct and re-test. **Whether the resulting architecture satisfies a regulatory obligation is a compliance/legal decision, not a technical one.**

6. Troubleshooting — Microsoft Purview Architecture & Design

The failure modes that make a Purview **architecture wrong, unenforceable, or drifted** — plus the discovery, RBAC, scoping, and naming issues you'll hit standing up the foundation guardrails. Each: **symptom** → **likely cause** → **fix.** Most "the design doesn't hold" reports are a licensing gap, an evolving role-group/portal surface, or drift from the naming standard — this guide separates those from real faults. Regulatory/records/legal **determinations** are out of scope here; escalate them.

`PURVARCH-Discovery.ps1` shows a capability as unavailable / "CHECK"

- **Cause A — not licensed.** Advanced DLP, auto-labeling, insider risk, communication compliance, Audit

Premium, DSPM, and adaptive scopes need **E5 / E5 Compliance** (some preview). The capability can't be designed *around* until it's licensed. **Fix:** confirm the SKU (Requirement 2.1) in the M365 admin center;

record the capability as **deferred — not licensed** on the `PA Capability Map – Tenant – Design` rather than inventing a workaround.

• **Cause B — missing read role.** The discovery account can't read that object type. **Fix:** assign the least-privilege **read** role for that domain (Requirement 2.2) and re-run.

• **Cause C — portal-only in your tenant.** Some capability surfaces have no stable cmdlet. **Fix:** treat it as portal-read/attested in the design; don't assert a cmdlet you can't confirm (`Get-Command`).

Role-group names in the design don't exist in the tenant

• **Cause:** Purview role-group names **evolve**, and many tenants now use **unified Purview RBAC** where the names/scoping differ from classic Security & Compliance role groups.

• **Fix:** `PURVARCH-Configuration.ps1` prints `[MISSING] role group not found` per unmatched name — do **not** guess. Confirm the current names in **Settings** → **Roles & scopes** → **Permissions** (verify path), update the `PA RBAC Model – Tenant – Design`, then re-run the dry-run before `-Commit`.

`PURVARCH-Configuration.ps1 -Commit` grants a broad role I didn't intend

• **Cause:** the design CSV contained an `Add` to a broad role (Organization Management, Compliance Administrator/Data Administrator); the script **warns** but honors the row.

• **Fix:** the `[WARN] Add to BROAD role` line is your gate — review it in the dry-run. Broad roles must be rare and deliberate (least privilege). **Rollback:** re-run with the reverse `Action` (Add ↔ Remove) for that row, then re-validate separation of duties.

Separation-of-duties conflicts appear in the RBAC validation

• **Cause:** a person administers **two** domains that must stay separate (e.g. insider risk **and** eDiscovery), or a reader also holds a broad admin role.

• **Fix:** `PURVARCH-Validation.ps1` emits the `PA RBAC SoD – Conflicts` register. Each conflict is **resolved** (remove one membership) or **accepted** with a recorded, named exception — do not leave it silent. Information protection ≠ insider risk ≠ eDiscovery ≠ records is the design intent.

The pilot ring group / adaptive scope won't create

• **Cause A — Graph permission.** Creating/populating `SG-PA-Pilot` needs **Group.ReadWrite.All**; a dry-run only needs read. **Fix:** consent the write scope for the `-Commit` run (Requirement 2.2).

• **Cause B — adaptive scope unlicensed.** `New-AdaptiveScope` is licensing-/tenant-dependent. **Fix:**

`PURVARCH-Set-Scope.ps1` prints `[CHECK] New-AdaptiveScope not available` when the cmdlet is absent — verify your plan; use a **static** scope in the design instead of inventing the capability.

- **Cause C — the example query doesn't match your directory.** The script's example scopes

`Department = PurviewPilot`. **Fix:** adjust the `FilterConditions` to an attribute your tenant actually populates before `-Commit`; the example is a scaffold, not a determination.

New capability objects don't conform to the naming standard

- **Cause:** an object was created in the portal without the `<Purpose> – <Scope> – <Stage>` convention

(`Sim`/`Prod`), so drift is invisible until audit.

- **Fix:** `PURVARCH-Validation.ps1` lists non-conforming names against the `PA Naming Standard – Tenant –

Design`. Renaming is per-object in the portal; route it through the roadmap remediation. The standard is what makes the simulation-first discipline enforceable across phases.

A capability policy was built without piloting in simulation/test

- **Cause:** a phase solution created a DLP/label/retention/insider-risk policy directly in **enforce**,

bypassing the architecture's pilot-first contract.

- **Fix:** this design solution builds **no** capability policy — each is owned by its own solution and

must pilot in simulation/test first against this architecture. Move the policy back to simulation/test/unpublished, validate on the pilot ring, then expand. The naming standard (`Sim` vs `Prod`) makes this catchable.

`Connect-IPPSSession` / `Connect-MgGraph` fails

- **Cause:** module not installed, or MFA/modern-auth not completed.
- **Fix:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` and `Install-Module Microsoft.Graph

-Scope CurrentUser`; complete the interactive modern-auth sign-in (Requirement 2.3). The discovery script reports which endpoints connected.

The design "drifts" between phases

- **Cause:** joiners/leavers change role membership, plans change what's licensed, or Microsoft ships

portal/RBAC changes that move a surface.

- **Fix:** re-run `PURVARCH-Validation.ps1` / `PURVARCH-Report.ps1` at each phase boundary and on a

cadence; treat the architecture as a **living document** — re-baseline periodically. Conformance of new objects to the RBAC model, scoping plan, and naming standard is the standing check.

When to escalate beyond this solution

• **Regulatory applicability, retention schedules, records file plans, and the lawful basis for insider-risk/communication monitoring → compliance, records, privacy, and legal**. The architecture carries these decisions; it does not make them.

• **A capability's actual policy build** → its **own solution**, against this design, piloted in simulation/test first.

• **A role-group or portal surface that has genuinely changed** → verify against current Microsoft documentation; Purview RBAC and the portal evolve, and unified RBAC may be in effect in your tenant.

Best Practices — Microsoft Purview Architecture & Design

Field-tested guidance for designing Purview before you scale it. It follows the same order the solution is built: get the **capability map and licensing** right, then the **RBAC and scoping guardrails**, then let the **phased roadmap** drive the build — deliberate, least-privilege, and defensible.

1. Map to licensing before you design a phase

• The most common Purview planning failure is designing a phase around a capability the plan does not license. **Confirm each capability is licensable first** (E3 vs E5 / E5 Compliance vs add-on vs preview), and mark the rest **deferred (not licensed)** on the map — do not design around them.

• Treat the script's capability-availability output as **indicative**; confirm exact entitlement with Microsoft licensing and by checking the capability appears in the portal.

2. Architect separation of duties from day one

• Separate the administration of **information protection, insider risk, communication compliance, eDiscovery, and records** into distinct role groups with distinct people. Retrofitting separation after everyone is a Compliance Administrator is expensive and audit-visible.

• Reserve **Global Administrator** and **Compliance Administrator**; prefer the **narrow, function-specific** role group. Keep readers/auditors read-only. Record and review SoD conflicts.

3. Design scoping as a first-class decision

• Choose **adaptive vs static** scope **per capability** deliberately. Adaptive scopes (query-based on user/site/group attributes) re-evaluate automatically and scale; static scopes are explicit and simple. Decide once, at design time, because every later policy inherits it.

• Use **administrative units** where scoped delegation matters (e.g. a region or business unit administers its own labels). Availability is licensing-gated — verify.

4. A pilot ring is what makes "pilot" real

• Define a named **pilot ring** (`SG-PA-Pilot`) before any capability is built. "Pilot" without a

defined population is a hope; with a ring it is a contained, reversible test on known users/sites.

• Expand **ring by ring**, re-running the gates after each expansion — never flip a high-impact policy

to enforce, tenant-wide, in one move.

5. Make simulation-first a designed pattern, not a habit

• Bake the pattern into the architecture: every high-impact policy (DLP, labels, retention, insider

risk) is created in **simulation/test/disabled** mode, named with the `Sim` suffix, scoped to the pilot ring, validated, then promoted to `Prod`. The naming standard and pilot ring make this **enforceable**, not just advisory.

6. Set the naming standard before the first object

• Names are effectively immutable once dozens of policies reference them. Adopt `<Purpose> – <Scope> –

<Stage> (with Sim/Prod`) **before** Phase 1, and apply it to every object. Report drift so nonconforming legacy objects are visible and planned for renaming.

7. Sequence phases — never run them all at once

• Follow **Foundation** → **Know** → **Protect** → **Prevent loss** → **Govern** → **Advanced**. Each phase depends on the

guardrails and classification the earlier phases establish; DLP without classification, or protection without roles, produces sprawl. Respect the entry/exit gate on each phase.

8. Keep the architecture a living document

• Re-baseline periodically and at every plan change. Review the RBAC model on **joiners/leavers**,

re-check **licensing** when subscriptions change, and watch for the ongoing **portal / unified Purview RBAC** evolution Microsoft ships — verify names and paths against your tenant rather than assuming.

9. Design carries determinations — it does not make them

• The architecture is the technical vessel for **regulatory applicability, retention schedules, records

file plans, and the lawful basis **for insider-risk/communication monitoring. Those remain** compliance / records / privacy / legal** determinations. Capture the owner per capability domain and route determinations to them.

10. Protect the privacy-sensitive domains in the design itself

• Insider Risk and Communication Compliance are HR/privacy-sensitive. Design **strict role separation**,

anonymization/pseudonymization where the capability supports it, and a recorded **lawful basis** into the architecture **before** those phases — not as a later add-on.

11. Verify against your tenant — the surfaces evolve

- Purview capability availability, role-group names, adaptive-scope support, and portal paths are

licensing-dependent and **actively changing**. Confirm capability availability and exact paths **against your licensing/tenant**; where a capability is in preview, treat it as such.

12. Keep the design artifacts and evidence intact

- The capability map, RBAC model, scoping plan, naming standard, and roadmap are the **defensible record**

of what Purview deploys and why. Version them, keep them current as phases complete, and treat the posture report as a point-in-time snapshot — the **authoritative** state is always the portal.

Deliverables Summary — Microsoft Purview Architecture & Design

Everything included in this solution, and what each piece is for.

Documentation

Document	Purpose
Solution Overview (`01-solution-overview.md`)	What the architecture delivers — the capability map, RBAC model, scoping strategy, naming standard, and phase roadmap — the phase model, the role-separation model, and the Assess→Design→Configure→Pilot→Validate→Expand approach.
Requirements (`02-requirements.md`)	Architecture-specific prerequisites — licensing baseline and what it gates, readable role groups + agreed separation, the in-scope capability/phase decision, scoping-strategy inputs (adaptive vs static, admin units, workloads), the naming standard, compliance/records/privacy/legal owners, the PowerShell/Graph toolchain, and a pilot ring with change control.
Implementation (`03-implementation.md`)	Ordered steps: baseline the tenant, map capabilities to licensing/phases, design the RBAC model + scoping strategy + naming standard, stand up the foundation guardrails, validate, then execute the roadmap phase by phase — with pass/fail gates and rollback.

Document	Purpose
Testing & Validation (`05-testing.md`)	Ten pass/fail gates from licensing and the capability map through RBAC separation, least privilege, scoping and the pilot ring, naming and simulation-first discipline, to the posture report and safe automation defaults.
Troubleshooting (`06-troubleshooting.md`)	The common design/foundation failure modes (a capability missing from the portal, role-group names that differ, SoD conflicts, adaptive scope unavailable, naming drift, connection/permission issues) and how to resolve them.
Best Practices (`07-best-practices.md`)	Map-to-licensing-first, architect separation of duties, scoping as first-class, the pilot ring, simulation-first as a designed pattern, naming before the first object, sequenced phases, living architecture, and privacy-by-design for the sensitive domains.
Executive Summary (`executive-summary.md`)	One-page business framing for decision-makers.

PowerShell tools (5)

All tools are **read-only or `-WhatIf` by default**, authenticate **interactively** (no credentials in code), and use **Security & Compliance PowerShell** (`Connect-IPPSSession`) and/or **Microsoft Graph** (`Connect-MgGraph`). The Purview architecture is **portal-managed**; these tools assess the estate, scaffold the RBAC and scope guardrails, validate the design, and report — they do not replace the portal.

Script	Type	What it does
`PURVARCH-Discovery.ps1`	Read-only	**Baseline / discovery**: reports subscribed SKUs and an **indicative** capability-availability mapping (via Graph), the current compliance **role groups** and their membership, existing **labels / DLP / retention policies** and **adaptive scopes**, and the **audit** state — the `PA Design Baseline – Tenant – Assess` snapshot. Changes nothing.

Script	Type	What it does
`PURVARCH-Configuration.ps1`	`-WhatIf` default	**RBAC-model configuration helper**: validates the designed role-group membership and composes `Add-RoleGroupMember` / `Remove-RoleGroupMember` to apply least-privilege, separated membership. **Runs with `-WhatIf` by default** and prints exactly what it would change; `-Commit` applies. Builds no capability policy — those are separate, piloted changes.
`PURVARCH-Validation.ps1`	Read-only	**Design-conformance checker**: reports separation-of-duties conflicts (`PA RBAC SoD – Conflicts`), least-privilege posture, capability-vs-licensing coverage, pilot-ring/adaptive-scope presence, and objects that do **not** conform to the naming standard — as pass/fail lines aligned to `05-testing.md`.
`PURVARCH-Report.ps1`	Read-only	**Design-posture rollup**: emits the `PA Capability Map – Tenant – Design` scaffold and the `PA Design Posture – Report` (capability coverage vs licensing, RBAC separation, scope coverage, naming conformance, phase status) to CSV; notes the portal is authoritative.
`PURVARCH-Set-Scope.ps1`	`-WhatIf` default	**Pilot-ring / scope helper**: previews and (with `-Commit`) creates/populates the pilot group `SG-PA-Pilot` and — where adaptive scopes are in the design — scaffolds the example adaptive scope `AS-PA-Pilot` (`New-AdaptiveScope`, where available), so expansion stays deliberate and reversible.

Named objects (verbatim — used across docs, scripts, and tests)

- **Tenant baseline snapshot:** `PA Design Baseline – Tenant – Assess`

- **Capability map:** `PA Capability Map – Tenant – Design`
- **RBAC model:** `PA RBAC Model – Tenant – Design`
- **RBAC separation-of-duties conflicts:** `PA RBAC SoD – Conflicts`
- **Scoping plan:** `PA Scoping Plan – Tenant – Design`
- **Naming standard:** `PA Naming Standard – Tenant – Design`
- **Phase roadmap:** `PA Phase Roadmap – Program – Design`
- **Design posture report:** `PA Design Posture – Report`
- **Pilot ring group:** `SG-PA-Pilot`
- **Example adaptive scope (pilot):** `AS-PA-Pilot`
- **Phases: 0 Foundation & Governance · 1 Know · 2 Protect · 3 Prevent loss · 4 Govern & retain · 5 Advanced (risk)**
- **Future object naming:** `<Purpose> – <Scope> – <Stage>` with `Sim` (simulation/test) and `Prod` (enforced)

What this solution deliberately does NOT do

- It does **not** build the capability policies (labels, DLP, retention, insider risk) — those are built in **their own solutions**, against this design, and **piloted in simulation/test first**.
- It does **not** enforce anything tenant-wide — the only things it configures are the **foundation guardrails** (least-privilege role membership and the pilot ring/scope), and those default to `-WhatIf`.
- It does **not** decide **which regulations apply**, the **retention schedule**, the **records file plan**, **or the** lawful basis **for insider-risk/communication monitoring — those are** compliance/records/privacy/legal** determinations.
- It does **not** invent portal paths, role-group names, or cmdlets — where a surface is portal-only or evolving, the tools stop at assessment/scaffolding and say "verify against your licensing/tenant".

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PURVARCH-Configuration.ps1

Purpose: Applies the "PA RBAC Model - Tenant - Design" by adding/removing compliance role-group members from a design file, enforcing least-privilege separation of duties. It composes Add-RoleGroupMember / Remove-RoleGroupMember. SAFE BY DEFAULT: it runs in -WhatIf mode and only DESCRIBES the change; nothing is written until -Commit is supplied. It configures ONLY the foundation RBAC guardrails — it builds NO capability policy (DLP, labels, retention, insider risk); those are separate, piloted-in-simulation changes owned by their own solutions. HOW TO RUN Dry-run (default) from a CSV of Group,Member,Action rows: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Configuration.ps1 -DesignCsv .\rbac-model.csv Apply after reviewing the dry-run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Configuration.ps1 -DesignCsv .\rbac-model.csv -Commit

How to run: Dry-run (default) from a CSV of Group,Member,Action rows: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Configuration.ps1 -DesignCsv .\rbac-model.csv Apply after reviewing the dry-run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Configuration.ps1 -DesignCsv .\rbac-model.csv -Commit

PURVARCH-Discovery.ps1

Purpose: Connects to Security & Compliance PowerShell (and, optionally, Microsoft Graph) to capture the Microsoft Purview design baseline BEFORE the architecture is scaled. It reports the subscribed SKUs it can read and an INDICATIVE capability-availability mapping, the current compliance role groups and their membership, existing sensitivity labels / DLP / retention policies and any adaptive scopes, and the unified-audit-log state. This is the "PA Design Baseline - Tenant - Assess" snapshot. Read-only — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVARCH-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\PURVARCH-Discovery.ps1 -IncludeGraph powershell -ExecutionPolicy Bypass -File .\PURVARCH-Discovery.ps1 -IncludeGraph -CsvPath .\baseline.csv

How to run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\PURVARCH-Discovery.ps1 -IncludeGraph powershell -ExecutionPolicy Bypass -File .\PURVARCH-Discovery.ps1 -IncludeGraph -CsvPath .\baseline.csv

PURVARCH-Report.ps1

Purpose: Produces the "PA Capability Map - Tenant - Design" scaffold and the "PA Design Posture - Report" — capability coverage vs licensing, RBAC role-group/member summary, existing policy/scope counts, and the phase model — as a leadership-facing CSV rollup. Read-only — it changes nothing. It reads the same signals as PURVARCH-Discovery.ps1 and structures them for reporting. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVARCH-Report.ps1 -CsvPath .\design-posture.csv powershell -ExecutionPolicy Bypass -File .\PURVARCH-Report.ps1 -IncludeGraph -CsvPath .\design-posture.csv

How to run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Report.ps1 -CsvPath .\design-posture.csv powershell -ExecutionPolicy Bypass -File .\PURVARCH-Report.ps1 -IncludeGraph -CsvPath .\design-posture.csv

PURVARCH-Set-Scope.ps1

Purpose: Stands up the foundation "pilot ring" the architecture designs: a security group (default SG-PA-Pilot) that every capability validates on first, and - where adaptive scopes are in the design - an example query-based adaptive scope (default AS-PA-Pilot). SAFE BY DEFAULT: it runs in -WhatIf mode and only DESCRIBES what it would create/populate; nothing is written until -Commit is supplied. It creates NO capability policy (DLP, label, retention, insider risk) - only the ring and scope those policies will later target, in simulation/test first. HOW TO RUN Dry-run (default) - report current state and what WOULD be created: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Set-Scope.ps1 Create/populate after reviewing the dry-run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Set-Scope.ps1 -PilotMemberCsv .\pilot-members.csv -Commit

How to run: Dry-run (default) - report current state and what WOULD be created: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Set-Scope.ps1 Create/populate after reviewing the dry-run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Set-Scope.ps1 -PilotMemberCsv .\pilot-members.csv -Commit

PURVARCH-Validation.ps1

Purpose: Validates that the Microsoft Purview design is conformant BEFORE capabilities are built against it. It reports separation-of-duties conflicts across the compliance role groups (the "PA RBAC SoD - Conflicts" register), least-privilege posture, whether the pilot ring / adaptive scope is present, and which existing objects do NOT conform to the "<Purpose> - <Scope> - <Stage>" naming standard. Read-only — it changes nothing. Findings are pass/fail lines aligned to 05-testing.md. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PURVARCH-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\PURVARCH-Validation.ps1 -PilotGroup SG-PA-Pilot -AdaptiveScope AS-PA-Pilot

How to run: powershell -ExecutionPolicy Bypass -File .\PURVARCH-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\PURVARCH-Validation.ps1 -PilotGroup SG-PA-Pilot -AdaptiveScope AS-PA-Pilot